STRIPE PROJECT

2. OLAP Data System

D8.2. OLAP SQL Queries Examples

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D8.2 : OLAP Queries / v22 --- 2026/11/05.*

- D8.2.1. Contexte
- D8.2.2. P1 : Traitement d'un paiement
  - P1.1. : Revenus mensuels nets des marchands vs. commission _Stripe_
  - P1.2 : Top-10 des produits par revenu (tous marchands confondus)
- D8.2.3. P2 : Évaluation du risque de fraude
  - P2.1 : Part des évaluations à niveau de risque bloquant, par marchand
  - P2.2 : Taux de fraude confirmée par niveau de risque évalué
- D8.2.4. P3 : Cycle de vie d'un abonnement
  - P3.1. Derniers revenus récurrents mensuels par marchand
  - P3.2. Clients à risque de désabonnement par segment
- D8.2.5. P4 : Accès et modification de données
  - P4.1. Tentatives d'accès non autorisées par acteur
  - P4.2. Historique d'accès à un client donné
- D8.2.6. P5 : Traitement d'une demande de droit
  - P5.1. Taux de respect des délais légaux par réglementation
  - P5.2. Demandes en cours proches de l'échéance légale
- D8.2.7. P6 : Traitement d'un incident de sécurité
  - P6.1. Taux de respect du délai de notification par sévérité
  - P6.2. Incidents de sécurité par marchand et par type
- D8.2.8. Agrégats et croisement de faits
  - A.1. Tableau de bord marchand : revenu des 30 derniers jours
  - A.2. Revenu et incidents de sécurité par marchand et par mois
- Références

D8.2.1. Contexte

Ce document propose quelques exemples de requêtes sur les données OLAP pour chacun des processus métiers (cf. ci-dessous). L'objectif est de vérifier que chaque processus est exploitable sans jointure OLTP. Quelques requêtes supplémentaires illustrent le requêtage sur des tables d'agrégats avec ou sans croisement des faits.

Processus métiers cibles (rappel)

- P1 : Traitement d'un paiement
- P2 : Évaluation du risque de fraude
- P3 : Cycle de vie d'un abonnement
- P4 : Accès et modification de données
- P5 : Traitement d'une demande de droit
- P6 : Traitement d'un incident de sécurité

Conventions

Dialecte

Dans ce document, *PostgreSQL* est le moteur de référence du modèle physique [D3-B]. Les expressions idiomatiques (`DISTINCT ON` , `to_char` , etc.) ne sont pas transcrites en SQL standard. Les transpositions des expressions SQL spécifiques vers un moteur colonne (Redshift, BigQuery, Snowflake, ClickHouse) reste à faire ([D3], D3.5.4).

Dimensions SCD2 (`DimMerchant` , `DimCustomer` , etc.)

Une ligne de fait désigne la version de la dimension en vigueur le jour de l'événement [D3-C] (ETL-01). Deux lectures sont possibles :

- *l'attribut tel qu'il était à l'événement* : jointure sur la clé de substitution du fait, sans filtre `is_current` ;
- *l'attribut actuel* (un libellé, par exemple) : jointure sur la clé de substitution du fait, puis jointure de la version courante par la clé métier (`merchant_id` , `actor_id`), notée `cur` dans les requêtes.

Dans les deux cas, les requêtes regroupent par clé métier, jamais par clé de substitution ni par libellé : un marchand a une `merchant_key` *par version*. Joindre le fait à `is_current = true` écarte tous les faits rattachés à une version antérieure.

Dates

Toutes les dates sont en UTC [D3-C] (2.C.2) : « aujourd'hui » s'écrit `(now() AT TIME ZONE 'UTC')::date` , jamais `CURRENT_DATE` , qui dépend du fuseau de la session. Les filtres de période portent sur `date_key` (borne calculée une fois, au format AAAAMMJJ), jamais sur une fonction appliquée à une colonne : l'élagage de partitions reste possible [D3] (D3.5.2, D3.5.3).

Transactions abouties

`FactTransaction` contient toutes les transactions, y compris échouées ou en attente. Les requêtes de revenu ne retiennent que les statuts `successful` et `refunded` , comme la définition de « transaction aboutie » de [D3-C] (ETL-10).

D8.2.2. P1 : Traitement d'un paiement

Faits utilisés : `FactTransaction` .

P1.1. : Revenus mensuels nets des marchands vs. commission *Stripe*

Consigne : récupérer les revenus mensuels nets des marchands (dans la devise de référence) et les commissions Stripe correspondantes, sur les 12 derniers mois civils (le mois en cours, partiel, compris).

```
SELECT
    d.year, d.month, m.merchant_id, cur.legal_name AS merchant,
    COUNT(*) AS transactions,
    SUM(ft.amount_eur) AS transactions_amount_euro,
    SUM(ft.merchant_net_revenue_eur) AS merchant_net_revenue_euro,
    SUM(ft.stripe_revenue_eur) AS stripe_revenue_euro
FROM FactTransaction ft
JOIN DimDate d ON d.date_key = ft.date_key
JOIN DimTransactionStatus ts ON ts.status_key = ft.status_key
JOIN DimMerchant m ON m.merchant_key = ft.merchant_key
JOIN DimMerchant cur ON cur.merchant_id = m.merchant_id AND cur.is_current
WHERE ts.status_code IN ('successful', 'refunded')
  AND ft.date_key >= to_char(date_trunc('month', now() AT TIME ZONE 'UTC') - INTERVAL '11 months',
'YYYYMMDD')::int
GROUP BY d.year, d.month, m.merchant_id, cur.legal_name
ORDER BY d.year, d.month, merchant_net_revenue_euro DESC;
```

La question centrale de BR2 [D0] : les deux vues (marchand/Stripe) cohabitent sur la même ligne de fait, pas besoin de recalculer l'une à partir de l'autre. Le nom affiché est le nom actuel du marchand, et les montants de toutes ses versions sont cumulés. Les rétrofacturations ne sont pas déduites du revenu net ([D3-C], ETL-21).

P1.2 : Top-10 des produits par revenu (tous marchands confondus)

Consigne : récupérer les 10 produits qui rapportent le plus (montant encaissé, en devise de référence), avec le détail par devise de transaction.

```
WITH revenue AS (
    SELECT
        p.product_id, p.name AS product, p.merchant_id,
        c.currency_code AS transaction_currency, c.decimals,
        x.fiscal_year, x.reference_rate,
        COUNT(*)            AS transactions,
        SUM(ft.amount_minor) AS amount_minor,
        SUM(ft.amount_eur)  AS amount_eur
    FROM FactTransaction ft
    JOIN DimProduct p  ON p.product_key = ft.product_key
    JOIN DimCurrency c ON c.currency_key = ft.currency_key
    JOIN DimReferenceExchangeRate x ON x.exchange_rate_key = ft.exchange_rate_key
    JOIN DimTransactionStatus ts ON ts.status_key = ft.status_key
    WHERE ts.status_code IN ('successful', 'refunded')
    GROUP BY p.product_id, p.name, p.merchant_id, c.currency_code, c.decimals,
             x.fiscal_year, x.reference_rate
```

```
),
ranked AS (
    SELECT *, SUM(amount_eur) OVER (PARTITION BY product_id) AS product_amount_eur
    FROM revenue
),
top_products AS (
    SELECT DISTINCT product_id, product_amount_eur
    FROM ranked
    ORDER BY product_amount_eur DESC, product_id
    LIMIT 10
)
SELECT
    r.product, mc.legal_name AS merchant, r.transaction_currency, r.transactions,
    ROUND(r.amount_minor / (10::numeric ^ r.decimals), r.decimals) AS amount_in_transaction_currency,
    r.fiscal_year, r.reference_rate,
    r.amount_eur AS amount_in_euros,
    r.product_amount_eur AS product_total_in_euros
FROM ranked r
JOIN top_products tp ON tp.product_id = r.product_id
LEFT JOIN DimMerchant mc ON mc.merchant_id = r.merchant_id AND mc.is_current
ORDER BY r.product_amount_eur DESC, r.product_id, r.amount_eur DESC;
```

Seule requête du document à montrer côte à côte le montant d'origine (`amount_minor` ramené à l'unité majeure par `DimCurrency.decimals`), le taux de référence qui a servi (`DimReferenceExchangeRate` , joint par `exchange_rate_key`) et le montant converti — démontre concrètement la comparabilité inter-devises. Le classement porte sur le produit (`product_id`), et non sur le couple produit-devise ni sur le nom, qui n'est pas unique entre marchands. Les transactions sans produit (`product_key` nul) n'y figurent pas.

D8.2.3. P2 : Évaluation du risque de fraude

Faits utilisés : `FactFraudEvent` .

P2.1 : Part des évaluations à niveau de risque bloquant, par marchand

Consigne : récupérer, par marchand, la part des évaluations de fraude dont le niveau de risque prévoit le blocage de la transaction (`DimFraudRiskLevel.blocks_transaction`).

```
SELECT
    m.merchant_id, cur.legal_name AS merchant,
    COUNT(*) AS fraud_events,
    COUNT(*) FILTER (WHERE frl.blocks_transaction) AS blocking_level_events,
    ROUND(100.0 * COUNT(*) FILTER (WHERE frl.blocks_transaction)
          / NULLIF(COUNT(*), 0), 2) AS blocking_level_percentage
FROM FactFraudEvent ffe
JOIN DimMerchant m ON m.merchant_key = ffe.merchant_key
JOIN DimMerchant cur ON cur.merchant_id = m.merchant_id AND cur.is_current
JOIN DimFraudRiskLevel frl ON frl.risk_level_key = ffe.risk_level_key
GROUP BY m.merchant_id, cur.legal_name
ORDER BY blocking_level_percentage DESC;
```

Le motif d'alerting opérationnel dominant. `blocks_transaction` est un attribut du niveau de risque (la règle), et non la décision effectivement appliquée à la transaction : `FactFraudEvent` ne porte pas cette décision. La requête mesure donc la part d'évaluations qui devraient être bloquées.

P2.2 : Taux de fraude confirmée par niveau de risque évalué

Consigne : récupérer les taux de fraudes confirmées par niveau de risque, pour les événements évalués il y a au moins 30 jours.

```
SELECT
    frl.label AS risk_level, frl.blocks_transaction,
    COUNT(*) AS fraud_events,
    COUNT(*) FILTER (WHERE ffe.is_confirmed_fraud) AS confirmed_frauds,
    ROUND(100.0 * COUNT(*) FILTER (WHERE ffe.is_confirmed_fraud)
          / NULLIF(COUNT(*), 0), 3) AS confirmation_rate_percentage
FROM FactFraudEvent ffe
JOIN DimFraudRiskLevel frl ON frl.risk_level_key = ffe.risk_level_key
WHERE ffe.date_key <= to_char((now() AT TIME ZONE 'UTC') - INTERVAL '30 days', 'YYYYMMDD')::int
```

```
GROUP BY frl.label, frl.blocks_transaction
ORDER BY confirmation_rate_percentage DESC;
```

Seule requête du document qui valide le modèle *a posteriori* plutôt que de l'exploiter en alerting immédiat — un besoin analytique distinct de P2.1. Une fraude se confirme des semaines après l'évaluation ([D3-C], ETL-32) : le délai de 30 jours écarte les événements trop récents pour être jugés (hypothèse à valider). La colonne `blocks_transaction` rappelle quels niveaux prévoient un blocage, dont le taux de confirmation se lit avec précaution.

D8.2.4. P3 : Cycle de vie d'un abonnement

Faits utilisés : `FactSubscriptionSnapshot`.

P3.1. Derniers revenus récurrents mensuels par marchand

Consigne : calculer la totalité des revenus récurrents mensuels des marchands rapportés par les abonnements actifs (statut `active`, [D3-C] ETL-42), d'après le dernier instantané de chaque abonnement.

```
WITH most_recent_snapshot AS (
    SELECT DISTINCT ON (subscription_id)
        subscription_id, merchant_key, mrr_eur, stripe_mrr_eur, is_active
    FROM FactSubscriptionSnapshot
    ORDER BY subscription_id, date_key DESC
)
SELECT
    m.merchant_id, cur.legal_name AS merchant,
    COUNT(*) FILTER (WHERE mrs.is_active) AS active_subscriptions,
    COALESCE(SUM(mrs.mrr_eur) FILTER (WHERE mrs.is_active), 0) AS merchant_mrr_in_euros,
    COALESCE(SUM(mrs.stripe_mrr_eur) FILTER (WHERE mrs.is_active), 0) AS stripe_mrr_in_euros
FROM most_recent_snapshot mrs
JOIN DimMerchant m ON m.merchant_key = mrs.merchant_key
JOIN DimMerchant cur ON cur.merchant_id = m.merchant_id AND cur.is_current
GROUP BY m.merchant_id, cur.legal_name
ORDER BY merchant_mrr_in_euros DESC;
```

Démontre le motif indispensable à ce type de fait (instantané périodique) : isoler le dernier état de chaque abonnement via `DISTINCT ON`, sans quoi toute agrégation mélangerait des instantanés historiques et actuels. Un abonnement qui a traversé un changement de version du marchand est compté une seule fois, sous le marchand (`merchant_id`) de son dernier instantané. Les abonnements `past_due`, facturables mais non actifs, n'entrent pas dans ce total.

P3.2. Clients à risque de désabonnement par segment

Consigne : récupérer le nombre d'abonnements à risque de désabonnement (statut qui déclenche la relance de paiement) par segment du client, d'après le dernier instantané de chaque abonnement.

```
WITH most_recent_snapshot AS (
    SELECT DISTINCT ON (subscription_id)
        subscription_id, customer_key, subscription_status_key, failed_attempt_count
    FROM FactSubscriptionSnapshot
    ORDER BY subscription_id, date_key DESC
)
SELECT
    COALESCE(c.segment, 'non défini') AS segment,
    COUNT(*) AS dunning_subscriptions,
    AVG(mrs.failed_attempt_count) AS average_failed_attempts
FROM most_recent_snapshot mrs
JOIN DimCustomer c ON c.customer_key = mrs.customer_key
JOIN DimSubscriptionStatus ss ON ss.subscription_status_key = mrs.subscription_status_key
WHERE ss.triggers_dunning = true
GROUP BY COALESCE(c.segment, 'non défini')
ORDER BY dunning_subscriptions DESC;
```

Exploite `DimSubscriptionStatus.triggers_dunning` pour une question de rétention client croisée avec la segmentation. Le segment est celui du client à la date du dernier instantané (version SCD2 du fait), et non son segment actuel. Les instantanés d'un abonnement sont à des dates différentes (une fin de période de facturation chacun) : seul « le dernier instantané de chaque abonnement » donne un état comparable, pas la date la plus récente de toute la table.

D8.2.5. P4 : Accès et modification de données

Faits utilisés : `FactAuditEvent`.

P4.1. Tentatives d'accès non autorisées par acteur

Consigne : récupérer le nombre de tentatives d'accès non autorisés de chaque acteur sur les 7 derniers jours UTC (aujourd'hui compris).

```
SELECT
    act.actor_id, cur.actor_type,
    COUNT(*) AS unauthorized_attempts,
    MAX(fae.event_timestamp) AS last_event_timestamp
FROM FactAuditEvent fae
JOIN DimActor act ON act.actor_key = fae.actor_key
JOIN DimActor cur ON cur.actor_id = act.actor_id AND cur.is_current
WHERE fae.was_authorized = false
  AND fae.date_key >= to_char((now() AT TIME ZONE 'UTC') - INTERVAL '6 days', 'YYYYMMDD')::int
GROUP BY act.actor_id, cur.actor_type
ORDER BY unauthorized_attempts DESC;
```

Le motif de sécurité dominant de cette table ; alerting sur les accès refusés. Le type affiché est le type actuel de l'acteur : un acteur réaffecté d'un type à un autre reste une seule ligne.

P4.2. Historique d'accès à un client donné

Consigne : récupérer l'historique des tentatives d'accès des acteurs pour un client donné (`:customer_id`).

```
SELECT
    fae.event_timestamp, act.actor_id, aa.label AS action,
    fae.resource_type, fae.was_authorized
FROM FactAuditEvent fae
JOIN DimCustomer c ON c.customer_key = fae.customer_key
JOIN DimActor act ON act.actor_key = fae.actor_key
JOIN DimAuditAction aa ON aa.audit_action_key = fae.audit_action_key
WHERE c.customer_id = :customer_id
ORDER BY fae.event_timestamp DESC;
```

Seule requête *paramétrée* du document. Répond directement à une obligation réglementaire concrète. Le filtre porte sur la clé métier `customer_id` : il couvre toutes les versions SCD2 du client. L'historique ne contient que les accès : les modifications (`ChangeAuditLog`) n'ont jamais de client ([D3-C] ETL-61).

D8.2.6. P5 : Traitement d'une demande de droit

Faits utilisés : `FactDataSubjectRequest`.

P5.1. Taux de respect des délais légaux par réglementation

Consigne : récupérer, par réglementation, le taux de demandes traitées dans le délai légal, pour les demandes reçues sur les 12 derniers mois civils (le mois en cours compris). Seules les demandes dont le respect du délai est connu (`is_within_deadline` renseigné) entrent dans le taux.

```
SELECT
    r.label AS regulation, r.response_deadline_days,
    COUNT(*) AS requests_assessed,
    COUNT(*) FILTER (WHERE fdsr.is_within_deadline) AS requests_within_deadline,
    ROUND(100.0 * COUNT(*) FILTER (WHERE fdsr.is_within_deadline)
          / NULLIF(COUNT(*), 0), 2) AS compliance_rate_percentage
FROM FactDataSubjectRequest fdsr
JOIN DimRegulation r ON r.regulation_key = fdsr.regulation_key
WHERE fdsr.is_within_deadline IS NOT NULL
  AND fdsr.received_date_key >= to_char(date_trunc('month', now() AT TIME ZONE 'UTC') - INTERVAL '11 months',
'YYYYMMDD')::int
GROUP BY r.label, r.response_deadline_days
ORDER BY compliance_rate_percentage ASC;
```

L'indicateur central du rapport de conformité RGPD/CCPA. Les demandes en cours ont un indicateur vide : les compter au dénominateur ferait baisser le taux sans qu'elles soient en retard (voir P5.2 pour celles-ci).

P5.2. Demandes en cours proches de l'échéance légale

Consigne : lister les demandes non clôturées, de la plus urgente à la moins urgente ; un nombre de jours restants négatif signale une demande en retard.

```
SELECT
    cur.legal_name AS merchant, fdsr.request_id, fdsr.request_type,
    d.full_date AS reception_date,
    d.full_date + r.response_deadline_days AS legal_deadline_date,
    d.full_date + r.response_deadline_days - (now() AT TIME ZONE 'UTC')::date AS remaining_days
FROM FactDataSubjectRequest fdsr
JOIN DimDate d ON d.date_key = fdsr.received_date_key
JOIN DimRegulation r ON r.regulation_key = fdsr.regulation_key
JOIN DimRequestStatus rs ON rs.request_status_key = fdsr.request_status_key
JOIN DimMerchant m ON m.merchant_key = fdsr.merchant_key
JOIN DimMerchant cur ON cur.merchant_id = m.merchant_id AND cur.is_current
WHERE rs.is_terminal = false
ORDER BY remaining_days ASC;
```

Seule requête du document à calculer une échéance plutôt que lire un indicateur déjà matérialisé ; dérive la date limite directement de `DimRegulation.response_deadline_days`.

D8.2.7. P6 : Traitement d'un incident de sécurité

Faits utilisés : `FactSecurityIncident`.

P6.1. Taux de respect du délai de notification par sévérité

Consigne : récupérer, par sévérité, le taux d'incidents notifiés dans les 72 heures. Un incident non notifié dont le délai est dépassé compte comme non conforme ; un incident non notifié dont le délai court encore est exclu du taux.

```
SELECT
    fsi.severity,
    COUNT(*) AS incidents,
    COUNT(*) FILTER (WHERE fsi.is_within_72h_deadline) AS notified_within_deadline,
    COUNT(*) FILTER (WHERE fsi.notified_date_key IS NOT NULL
                       AND NOT fsi.is_within_72h_deadline) AS notified_late,
    COUNT(*) FILTER (WHERE fsi.notified_date_key IS NULL
                       AND d.full_date + 3 < (now() AT TIME ZONE 'UTC')::date) AS not_notified_overdue,
    ROUND(100.0 * COUNT(*) FILTER (WHERE fsi.is_within_72h_deadline)
          / NULLIF(COUNT(*) FILTER (WHERE fsi.notified_date_key IS NOT NULL
                                       OR d.full_date + 3 < (now() AT TIME ZONE 'UTC')::date), 0), 2) AS
compliance_rate_percentage,
    ROUND(AVG(fsi.detection_to_notification_hours), 1) AS average_notification_time_hours
FROM FactSecurityIncident fsi
JOIN DimDate d ON d.date_key = fsi.detected_date_key
GROUP BY fsi.severity
ORDER BY compliance_rate_percentage ASC;
```

Pendant direct de P5.1 pour les incidents : même logique de conformité mais obligation réglementaire différente (72h de notification plutôt que délai de réponse). Ne retenir que les incidents notifiés surestimerait la conformité, car les non notifiés sont les cas les plus à risque. Le fait ne porte que des dates : un incident non notifié est jugé en retard quand `detected_date + 3 jours` est strictement passé, ce qui ne laisse aucun doute sur le dépassement des 72 heures. Le modèle ne distingue pas « notification non requise » de « pas encore notifié » ([D3-C], 2.C.3.8, O-09) : un incident sans notification requise est ici compté en retard.

P6.2. Incidents de sécurité par marchand et par type

Consigne : récupérer les incidents de sécurité affectés par marchand et par type d'incident ; les incidents transverses à la plateforme (sans marchand) sont regroupés à part.

```
SELECT
    COALESCE(cur.legal_name, 'transversal platform') AS merchant,
    fsi.incident_type,
    COUNT(*) AS incidents,
    SUM(fsi.affected_record_count) AS total_affected_records
FROM FactSecurityIncident fsi
LEFT JOIN DimMerchant m ON m.merchant_key = fsi.merchant_key
LEFT JOIN DimMerchant cur ON cur.merchant_id = m.merchant_id AND cur.is_current
GROUP BY m.merchant_id, cur.legal_name, fsi.incident_type
ORDER BY total_affected_records DESC NULLS LAST;
```

Seule mesure réellement additive du volet de conformité (`affected_record_count`). Les autres indicateurs de ces deux processus sont des délais, qui ne sont pas sommables entre eux. La jointure externe conserve les incidents sans marchand (`merchant_key` nul).

D8.2.8. Agrégats et croisement de faits

Faits et agrégats utilisés : `AggRevenueDailyMerchant` , `FactTransaction` , `FactSecurityIncident` .

A.1. Tableau de bord marchand : revenu des 30 derniers jours

Consigne : pour un marchand donné (`:merchant_id`), afficher jour par jour le nombre de transactions, le volume encaissé, le revenu net et les remboursements des 30 derniers jours UTC (aujourd'hui compris), sans lire le fait atomique.

- Paramètres : l'identifiant du marchand (`:merchant_id`).

```
SELECT
    d.full_date,
    SUM(a.transaction_count) AS transactions,
    SUM(a.gross_volume_eur) AS gross_volume_euro,
    SUM(a.merchant_net_revenue_eur) AS merchant_net_revenue_euro,
    SUM(a.refund_total_eur) AS refund_total_euro,
    ROUND(SUM(a.gross_volume_eur) / NULLIF(SUM(a.transaction_count), 0), 4) AS avg_ticket_euro
FROM AggRevenueDailyMerchant a
JOIN DimMerchant m ON m.merchant_key = a.merchant_key
JOIN DimDate d ON d.date_key = a.date_key
WHERE m.merchant_id = :merchant_id
  AND a.date_key >= to_char((now() AT TIME ZONE 'UTC') - INTERVAL '29 days', 'YYYYMMDD')::int
GROUP BY d.full_date
ORDER BY d.full_date;
```

Démontre la stratégie d'agrégation de [D3] (D3.4) : une ligne par jour et par marchand, lue directement. Le filtre porte sur la clé métier `merchant_id` , donc sur toutes les versions du marchand. Le panier moyen est recalculé à partir des sommes : la moyenne de moyennes `avg_ticket_eur` serait fausse. Les jours sans transaction n'ont pas de ligne.

A.2. Revenu et incidents de sécurité par marchand et par mois

Consigne : croiser, pour chaque marchand et pour chaque mois des 12 derniers mois civils, la commission perçue par Stripe et les incidents de sécurité détectés.

```
WITH revenue AS (
    SELECT m.merchant_id, d.year, d.month,
           SUM(ft.stripe_revenue_eur) AS stripe_revenue_euro
    FROM FactTransaction ft
    JOIN DimDate d ON d.date_key = ft.date_key
    JOIN DimMerchant m ON m.merchant_key = ft.merchant_key
    JOIN DimTransactionStatus ts ON ts.status_key = ft.status_key
    WHERE ts.status_code IN ('successful', 'refunded')
      AND ft.date_key >= to_char(date_trunc('month', now() AT TIME ZONE 'UTC') - INTERVAL '11 months',
'YYYYMMDD')::int
    GROUP BY m.merchant_id, d.year, d.month
),
incidents AS (
    SELECT m.merchant_id, d.year, d.month,
           COUNT(*) AS incidents,
           SUM(fsi.affected_record_count) AS affected_records
    FROM FactSecurityIncident fsi
    JOIN DimDate d ON d.date_key = fsi.detected_date_key
    JOIN DimMerchant m ON m.merchant_key = fsi.merchant_key
    WHERE fsi.detected_date_key >= to_char(date_trunc('month', now() AT TIME ZONE 'UTC') - INTERVAL '11 months',
'YYYYMMDD')::int
    GROUP BY m.merchant_id, d.year, d.month
)
SELECT
    cur.legal_name AS merchant,
    COALESCE(r.year, i.year) AS year, COALESCE(r.month, i.month) AS month,
    COALESCE(r.stripe_revenue_euro, 0) AS stripe_revenue_euro,
    COALESCE(i.incidents, 0) AS incidents,
    COALESCE(i.affected_records, 0) AS affected_records
FROM revenue r
FULL JOIN incidents i
```

```
       ON i.merchant_id = r.merchant_id AND i.year = r.year AND i.month = r.month
JOIN DimMerchant cur
       ON cur.merchant_id = COALESCE(r.merchant_id, i.merchant_id) AND cur.is_current
ORDER BY merchant, year, month;
```

Croisement de deux faits par leurs dimensions conformées (`DimMerchant` , `DimDate`) [D3-A] (2.A.6) : chaque fait est agrégé séparément au grain (marchand, mois), puis les résultats sont joints. Joindre les deux faits ligne à ligne multiplierait les montants par le nombre d'incidents. Les incidents sans marchand (`merchant_key` nul) n'y figurent pas.

Références

Spécifications

- [D0] Stripe Project Introduction

Livrables

- [D3] OLAP System Schema Design

Annexes

- [D3-A] OLAP Kimball Analysis
- [D3-B] OLAP Physical Data Model
- [D3-C] OLAP Conventions and Transformation Rules